

Semaine 1 — Cadrage + socle “recettes” (SANS IA)

Objectifs

- Définir clairement le **problème**
- Séparer **reconnaissance du plat** et **gestion des recettes**
- Avoir une base fonctionnelle **avant toute IA**

Ce que vous avez fait concrètement

- Définition d'un **format unique de recette** en `recipes.json`
- Structure claire :
 - Dish (id, nom, portions par défaut)
 - ingrédients (nom, quantité, unité)
 - étapes
- Implémentation du module :
 - `RecipeDB.load()`
 - `find_dish()`, `suggest()`
- Implémentation du **scaling automatique des quantités**
- Création d'un **CLI testable**

Livrable Semaine 1

```
python -m app.cli.get_recipe --dish pizza --servings 2
```

 Affiche une recette propre avec **quantités recalculées**

 **IA = 0**, mais **socle solide**

Semaine 2 — Compréhension du Deep Learning (fondations)

 Cette semaine est souvent oubliée dans les rapports, **mais elle est essentielle.**

Objectifs

- Comprendre **ce qu'est un réseau de neurones**
- Comprendre **la notion de frontière de décision**
- Valider que PyTorch fonctionne

Ce que vous avez fait

- Génération de données artificielles (points 2D)
- Classification linéaire / non linéaire
- XOR (cas impossible sans non-linéarité)
- Visualisation des frontières de décision
- Entraînement + convergence + accuracy

Livrable Semaine 2

- Scripts `classifier_points.py`, `xor_classifier.py`
- Figures montrant la séparation des classes
- Compréhension claire :

“Un réseau apprend une frontière de décision dans l’espace des données.”

 Très bon point à expliquer à l’oral

Semaine 3 — Classification d’images (MNIST → CNN)

Objectifs

- Passer du “jouet” à la **vraie image**
- Comprendre CNN, convolution, entraînement image

Ce que vous avez fait

- Chargement de MNIST avec `torchvision`
- Visualisation des images
- Entraînement d’un CNN
- Sauvegarde du modèle
- Test accuracy très élevée (>98 %)

Livrable Semaine 3

```
python app/week3/mnist_train.py
```

 Reconnaissance de chiffres manuscrits

 Validation complète de la chaîne image → label

Semaine 4 — Reconnaissance de plats (Food-101)

Objectifs

- Appliquer le deep learning à un **problème réel**
- Utiliser le **transfer learning**

Ce que vous avez fait

- Choix du dataset **Food-101**
- Sélection réaliste de **10 plats**
- Script de préparation du dataset :
 - `train / val / test`

- Entraînement avec **ResNet18 pré-entraîné**
- Fine-tuning progressif (layer4 + fc)
- Data augmentation
- Résultats :
 - **~93 % accuracy test**
 - top-3 très fiable

Livable Semaine 4

```
python app/week4/food_train.py
python app/week4/food_testset_eval.py
```

👉 Cœur IA du projet validé

Semaine 5 — Pipeline complet (photo → plat → recette)

Objectifs

- Relier **IA + recettes**
- Gérer l'incertitude

Ce que vous avez fait

- Script `predict_to_recipe.py`
- Prédiction top-3 + confiance
- Seuil de confiance (`min_conf`)
- Si incertain → pas de recette affichée
- Mapping strict :
 - `label IA == id recette`

Livable Semaine 5

```
python -m app.week4.predict_to_recipe --image photo.jpg --servings 4
```

👉 Pipeline terminal complet et robuste

Semaine 6 — Interface PyQt (MVP solide)

Objectifs

- Créer l'application demandée par le prof
- Pas de gadget, mais **stable**

✅ Ce que vous avez fait

- Interface PyQt fonctionnelle
- Chargement d'image
- Preview image
- Top-3 + confiance
- Slider portions
- Seuil de confiance réglable
- Affichage dynamique de la recette

📷 Le screenshot que tu as envoyé = preuve parfaite

📦 Livrable Semaine 6

```
python -m app.week5.gui
```

👉 Application prête pour la soutenance

📅 Semaine 7–8 — Rapport, soutenance, bonus (optionnels)

👉 Vous êtes **déjà prêts**, mais vous pouvez mentionner comme perspectives :

- Ajout de variantes de recettes
- Recherche par similarité (embeddings)
- Export PDF / TXT
- Extension à plus de plats

⚠️ À mentionner comme “travaux futurs”, pas comme fait.

🎯 Objectif du projet

L'objectif de ce projet est de concevoir une **application de deep learning** capable de :

- ⚠ Important :
- 👉 Le projet **ne cherche pas** à deviner les ingrédients exacts depuis l'image (problème trop complexe).
 - 👉 L'approche retenue est :

```

Image
↓
Réseau de neurones (ResNet18)
↓
Classe du plat (pizza, ramen, burger, ...)
↓
Base de recettes structurée (JSON)
↓
Recette + quantités ajustées
↓
Interface graphique PyQt

```

```
AM1_projet/
├── app/
│   └── core/                # Logique métier (recettes, scaling)
```

```
├── ┌── recipes.py
│   ├── scaling.py
│   └── units.py
├── week2/           # Fondations deep learning (points, XOR)
├── week3/           # MNIST (CNN chiffres)
├── week4/           # Food-101 (classification plats)
├── week5/           # Interface PyQt
│   ├── predictor.py
│   └── gui.py
├── data/
│   ├── recipes.json      # Base de recettes
│   └── food101_10/       # Dataset Food-101 (10 plats)
│       ├── train/
│       ├── val/
│       └── test/
├── models/
│   └── model_food.pth    # Modèle entraîné
├── requirements.txt
├── README.md
└── .venv/               # Environnement virtuel Python
```

Technologies utilisées

- **Python 3.10+**
 - **PyTorch** (deep learning)
 - **Torchvision** (datasets & modèles)
 - **ResNet18** (transfer learning)
 - **Food-101** (dataset d'images réelles)
 - **PyQt6** (interface graphique)
 - **PIL / Matplotlib**
 - **tqdm** (barres de progression)
-

Installation complète (pas à pas)

1 Cloner le projet

```
git clone <URL_DU_DEPOT>
cd AM1_projet
```

2 Créer et activer l'environnement virtuel

macOS / Linux

```
python3 -m venv .venv
source .venv/bin/activate
```

Windows (PowerShell)

```
python -m venv .venv  
.venv\Scripts\activate
```

Installer les dépendances Python

```
pip install --upgrade pip  
pip install -r requirements.txt
```

requirements.txt

```
torch  
torchvision  
matplotlib  
pillow  
tqdm  
rich  
PyQt6
```

 Si PyQt6 pose problème sur macOS :

```
pip install --no-cache-dir --default-timeout=120 --retries 30 PyQt6
```

Dataset images (Food-101)

Télécharger Food-101

- https://www.vision.ee.ethz.ch/datasets_extra/food-101/

Dézipper dans :

```
~/Downloads/food-101
```

Préparer un sous-ensemble (10 plats)

```
python app/week4/prepare_food101_subset.py \  
  --food101 ~/Downloads/food-101 \  
  --out data/food101_10 \  
  --mode symlink
```

Plats utilisés :

- pizza
- hamburger
- ramen
- spaghetti_bolognese
- caesar_salad
- fried_rice

- omelette
- sushi
- ice_cream
- chocolate_cake

Entraîner le modèle de reconnaissance de plats

```
python app/week4/food_train.py \  
  --data data/food101_10 \  
  --epochs 15 \  
  --batch_size 16 \  
  --lr 1e-4
```

 Résultats typiques :

- Accuracy test $\approx$ **93 %**
- Top-3 très fiable

Évaluation :

```
python app/week4/food_testset_eval.py \  
  --model models/model_food.pth \  
  --data data/food101_10
```

Base de recettes

Les recettes sont stockées dans :

data/recipes.json

Chaque plat contient :

- id (doit correspondre au label du modèle)
- servings_default
- ingrédients (nom, quantité, unité)
- étapes

Les quantités sont **recalculées dynamiquement** selon le nombre de personnes.

Pipeline terminal (sans interface)

```
python -m app.week4.predict_to_recipe \  
  --image data/food101_10/test/pizza/1001116.jpg \  
  --servings 4 \  
  --min_conf 0.70
```


Comportement :

- Si confiance $\geq$ seuil $\rightarrow$ recette affichée
 - Sinon $\rightarrow$ message “incertain” + top-3
-

Lancer l'application graphique (PyQt)

```
python -m app.week5.gui
```

Fonctionnalités :

-  Choix d'une image
 -  Prévisualisation
 -  Top-3 prédictions + confiance
 -  Slider nombre de personnes
 -  Seuil de confiance réglable
 -  Recette dynamique
-

Organisation pédagogique du projet

Semaine	Contenu
S1	Base de recettes + scaling
S2	Réseaux de neurones simples (points, XOR)
S3	CNN sur MNIST
S4	Food-101 + transfer learning
S5	Pipeline complet
S6	Interface PyQt
S7–8	Rapport, soutenance, bonus

Limites connues

- Le modèle reconnaît le **type de plat**, pas les ingrédients exacts
 - Sensible à :
 - mauvaise lumière
 - plat non centré
 - fond trop chargé
 - Dataset limité à 10 plats (choix volontaire)
-

Perspectives possibles

- Recherche par similarité (embeddings)
- Variantes de recettes par plat
- Export PDF / TXT
- Extension à plus de plats



Travail de groupe

Projet réalisé à **4 personnes**

Chaque membre a contribué sur :

- apprentissage deep learning
- dataset
- modèle
- pipeline
- interface

Les commits Git reflètent la participation individuelle.